BÀI TẬP THỰC HÀNH OOP
HỆ THỐNG QUẢN LÝ BÃI ĐỖ XE – GIAI ĐOẠN 2

Giảng viên: Nguyễn Hạnh Phúc - FIT - VMU
Ngôn ngữ lập trình: C++ (Chuẩn C++11 trở lên)
Chủ đề: Mảng tĩnh làm thuộc tính Lớp (Fixed Member Arrays), Quản lý Kích thước Vật lý vs Kích thước Logic, Thuật toán Dồn mảng và Xử lý Trùng lặp.

1 MỤC TIÊU & YÊU CẦU KỸ THUẬT

- **Mục tiêu:** Nâng cấp hệ thống quản lý bãi đỗ xe từ Giai đoạn 1 bằng cách **thay thế toàn bộ các biến đối tượng lẻ (`vehicle1, vehicle2, vehicle3` và `lot1, lot2, lot3`) bằng các Mảng tĩnh (Fixed-size Arrays).**
- **Hạn chế bắt buộc (Strict Constraints):**
  1. **ĐÓNG GÓI VÀ MẢNG TĨNH:** Tất cả mảng lưu trữ phải là thuộc tính `private` có kích thước cố định (`C-style array` hoặc `std::array`).
  2. **QUẢN LÝ KÍCH THƯỚC LOGIC:** Phải quản lý song song hai khái niệm:
     - **Sức chứa vật lý (Physical Capacity):** Kích thước khai báo tối đa của mảng (`MAX_VEHICLES = 3`, `MAX_LOTS = 10`).
     - **Kích thước logic (Logical Size / `currentCount`):** Số lượng phần tử thực tế đang có trong mảng.
  3. **THUẬT TOÁN DỒN MẢNG:** Khi xóa một xe hoặc một bãi đỗ, bắt buộc phải **dồn dịch các phần tử phía sau sang trái** để danh sách dữ liệu trong mảng luôn liên tục từ chỉ số `0` đến `currentCount - 1`.
  4. **CHƯA SỬ DỤNG CON TRỎ VÀ BỘ NHỚ ĐỘNG:** Tuyệt đối **KHÔNG** dùng con trỏ (`*`), cấp phát động (`new/delete`), hay `std::vector` (các nội dung này sẽ triển khai ở Giai đoạn 3 và Giai đoạn 5).

2 CẤU TRÚC VÀ THIẾT KẾ NÂNG CẤP CÁC LỚP (CLASS SPECIFICATIONS)

2.1 Lớp 1: `Vehicle` (Giữ nguyên cấu trúc đóng gói từ Giai đoạn 1)

- **Thuộc tính (`private`):** `vehicleID`, `vehicleType`, `entryTime`, `parkingDuration`, `parkingFee`.
- **Phương thức (`public`):** Getters/Setters, `calculateFee()`, `display()`.

2.2 Lớp 2: `ParkingLot` (Nâng cấp thuộc tính lưu trữ thành Mảng tĩnh)

- **Thuộc tính** (`private`)**:**
  - `int parkID`: Mã định danh bãi đỗ xe.
  - `char size`: Kích thước bãi đỗ (`'S'` - Nhỏ [1 xe], `'M'` - Trung bình [2 xe], `'L'` - Lớn [3 xe]).
  - `static const int MAX_VEHICLES = 3`: Sức chứa vật lý tối đa của một bãi đỗ.
  - `Vehicle vehicles[MAX_VEHICLES]`: **Mảng tĩnh chứa các đối tượng** `Vehicle`**.**
  - `int currentVehicleCount`: Biến đếm số lượng xe thực tế đang đỗ (Kích thước logic, khởi tạo = `0`).
- **Phương thức** (`public`)**:**
  - Constructor mặc định và Constructor theo `parkID`, `size`.
  - `bool isFull() const`: Kiểm tra bãi đã đạt sức chứa tối đa tương ứng với `size` chưa.
  - `bool addVehicle(const Vehicle& v)`: Thêm xe vào vị trí `vehicles[currentVehicleCount]` và tăng `currentVehicleCount` lên 1.
  - `bool removeVehicle(int vID)`: Tìm xe theo ID trong mảng, xóa xe và **dồn dịch mảng** các phần tử phía sau sang trái, giảm `currentVehicleCount` đi 1.
  - `int findVehicleIndex(int vID) const`: Hàm bổ trợ trả về chỉ số (index) của xe trong mảng (trả về `-1` nếu không tìm thấy).
  - `void displayLotInfo() const`: Duyệt mảng từ `0` đến `currentVehicleCount - 1` để in thông tin bãi và danh sách xe.

2.3 Lớp 3: `SystemInterface` (Mở rộng quy mô quản lý 10 Bãi đỗ xe bằng Mảng)

- **Thuộc tính** (`private`)**:**
  - `static const int MAX_LOTS = 10`: Mở rộng từ 3 bãi lên **tối đa 10 bãi đỗ xe.**
  - `ParkingLot parkingLots[MAX_LOTS]`: **Mảng tĩnh chứa các đối tượng** `ParkingLot`**.**
  - `int currentLotCount`: Số lượng bãi đỗ hiện có trong hệ thống (khởi tạo = `0`).
- **Phương thức** (`public`)**:**
  - `void runMenu()`: Điều khiển menu tương tác người dùng và xử lý ngoại lệ đầu vào khéo léo.
  - `bool isParkIDExists(int pID) const`: Kiểm tra ID bãi đỗ có bị trùng lặp trên mảng `parkingLots` hay không.
  - `bool isVehicleIDExists(int vID) const`: Duyệt qua toàn bộ các bãi đỗ trong mảng để kiểm tra ID xe có bị trùng lặp hay không.

– `int generateRandomID()`: Hàm phụ trợ tự động sinh ID ngẫu nhiên bằng `rand()` khi người dùng nhập trùng ID.

3 CÁC QUY TẮC BỔ SUNG & XỬ LÝ MẢNG NÂNG CẤP

1. **Quy tắc xếp xe tự động vào Mảng Bãi đỗ:**
   - Khi thêm một xe mới, hệ thống tự động tìm bãi đỗ **đầu tiên còn vị trí trống** trong mảng `parkingLots` để xếp xe vào. Nếu tất cả các bãi đỗ hiện có đều đã đầy, thông báo cảnh báo ra màn hình.
2. **Quy tắc xử lý Trùng lặp ID (Duplicate ID Handling):**
   - **Trùng ID Bãi đỗ:** Nếu người dùng nhập `parkID` đã tồn tại trong mảng `parkingLots`, hiển thị cảnh báo và **tự động gán một ID ngẫu nhiên** cho bãi đỗ đó.
   - **Trùng ID Xe:** Nếu người dùng nhập `vehicleID` đã tồn tại ở bất kỳ bãi đỗ nào trong hệ thống, hiển thị cảnh báo và **tự động gán một ID ngẫu nhiên** cho xe đó.
3. **Quy tắc Kiểm tra tính hợp lệ dữ liệu (Input Validation):**
   - Nếu người dùng nhập sai kiểu dữ liệu (ví dụ: nhập chữ vào biển số), chương trình phải bắt lỗi `std::cin.fail()`, xóa cờ lỗi `std::cin.clear()`, loại bỏ bộ đệm `std::cin.ignore()` và yêu cầu nhập lại một cách an toàn.

4 DANH SÁCH CHỨC NĂNG MENU (MAIN MENU)

Chương trình hiển thị menu 9 chức năng chính (chưa tích hợp Đọc/Ghi File, chức năng này sẽ hoàn thiện ở bài kết thúc):

1. **Tạo Bãi đỗ xe mới:** Kiểm tra mảng `parkingLots` đã đạt `MAX_LOTS` (10 bãi) chưa. Nếu chưa, nhận `parkID` và `size`, kiểm tra trùng ID để gán ID ngẫu nhiên nếu cần.
2. **Thêm xe vào bãi đỗ:** Nhập `vehicleID`, `vehicleType`, `entryTime`, `parkingDuration`. Kiểm tra trùng ID xe toàn hệ thống. Tự động xếp xe vào bãi đỗ còn chỗ trống đầu tiên trong mảng.
3. **Xóa một Bãi đỗ xe:** Nhập `parkID`, tìm vị trí trong mảng `parkingLots`, xóa và **dồn mảng bãi đỗ**.
4. **Lấy xe ra khỏi bãi đỗ:** Nhập `vehicleID`, duyệt mảng tìm xe, xóa xe và **dồn mảng xe bên trong bãi đỗ tương ứng**.
5. **Hiển thị danh sách tất cả các Bãi đỗ xe:** Duyệt mảng `parkingLots` từ `0` đến `currentLotCount - 1` để in tổng quan.
6. **Hiển thị danh sách xe theo bãi cụ thể:** Nhập `parkID`, duyệt mảng xe của bãi đó để in thông tin chi tiết.
7. **Liệt kê xe theo loại phương tiện:** Duyệt qua tất cả các bãi đỗ trong mảng `parkingLots` để lọc các xe khớp với `vehicleType` yêu cầu.

8. **Tóm tắt phí đỗ xe trung bình:** Lặp qua toàn bộ mảng dữ liệu để tính toán phí trung bình theo loại xe trên toàn hệ thống và phân chia theo từng bãi.
9. **Thoát chương trình.**